Tests d'intégration et recette fonctionnelle

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Tester un produit numérique **3**

A. Plan de tests 3

B. Types de tests 4

1. Tests unitaires....4

2. Tests d'intégration....4

3. Tests de recette fonctionnelle....4

a. Quelques définitions....5

II. Exercice : Quiz **6**

III. Tester en prenant en compte le cycle de développement **7**

A. Cycle en V 7

B. Méthodes Agiles 8

C. Préparation des tests lors des phases d'expression de besoin 10

D. Préparation des tests lors de l'écriture des spécifications fonctionnelles 10

E. Préparation des tests lors de la conception 10

IV. Exercice : Quiz **11**

V. Focus sur les tests d'intégration **12**

A. Stratégies du test d'intégration 12

B. Stratégie du Big Bang 13

C. Stratégie par lots fonctionnels 13

D. Stratégie incrémentale....13

VI. Exercice : Quiz **13**

VII. Focus sur la recette fonctionnelle **14**

A. Spécifications fonctionnelles....14

B. Plan de test 15

C. Manuel utilisateur 15

VIII. Exercice : Quiz **16**

IX. Essentiel **17**

X. Auto-évaluation **17**

A. Exercice : 17

B. Test....17

Solutions des exercices **18**

Tous droits réservés à STUDI - Reproduction interdite

I. Tester un produit numérique

Contexte

Les logiciels informatiques sont devenus omniprésents et indispensables à notre vie quotidienne. Les sites internet, les transactions bancaires, nos smartphones, nos voitures, tous utilisent des logiciels informatiques. Bien que le dysfonctionnement d'un site internet puisse être désagréable, les conséquences d'un dysfonctionnement logiciel sur un véhicule pourraient être dramatiques. Ces logiciels nécessitent généralement des millions de lignes de code et il pourrait suffire qu'un point « . » soit écrit à la place d'une virgule « , » pour rendre inopérant un système complexe !

Un logiciel doit bien sûr réaliser ce pourquoi il a été conçu, mais il ne doit pas faire ce pourquoi il n'a pas été conçu. Créé par l'homme, sur la base d'une expression de besoin plus ou moins précise, les logiciels sont soumis aux interprétations et aux aléas de la compréhension humaine, du développeur et de l'équipe qui l'entoure. Les logiciels contiennent des erreurs, c'est une certitude. Le développement d'un logiciel est comme toute activité humaine, sujet à des imperfections qui peuvent être sans conséquence dans certains cas, mais catastrophiques dans d'autres. Il faut donc s'efforcer de détecter les erreurs au plus tôt, car plus une erreur est détectée tardivement, et plus elle coûte chère à réparer !

A. Plan de tests

Le plan de test décrit les activités à mettre en œuvre pour tester un produit. Il peut évidemment s'appuyer sur l'analyse de risques du projet et le dossier de sécurité du projet (SSI). Le but est d'apporter un soin particulier aux aspects réputés sensibles. Le plan de tests doit répondre aux interrogations suivantes :

1. **Pourquoi tester ?**

   Identifier les enjeux et les objectifs à atteindre.

2. **Que faut-il tester ?**

   Préciser le périmètre des tests, ce qui est inclus et ce qui est exclu (et pourquoi).

3. **Comment le tester ?**

   Spécifier les types de tests, les méthodes, les outils, les ressources matérielles nécessaires. Mentionnez le scénario à appliquer et le résultat attendu. Ceci permet d'analyser les écarts.

4. **Qui mène les tests ?**

   Identifiez clairement les acteurs et leurs responsabilités aux différents niveaux (responsable des tests, rédacteurs, testeurs, debuggers, etc.).

5. **Quel est le calendrier des tests ?**

   Planifiez le déroulement des tests afin de ne pas être pris au dépourvu.

6. **Quels sont les livrables attendus ?**

   Mentionnez ce que vous attendez et à quel moment : des cahiers de recette, des scripts, des données, des rapports, des procès-verbaux.

Tous droits réservés à STUDI - Reproduction interdite

B. Types de tests

Plusieurs types de tests sont réalisés sur les produits issus de projets numériques. On trouve *généralement* dans l'ordre :

1. Tests unitaires

Les tests unitaires permettent de vérifier le bon fonctionnement d'une partie précise du logiciel, comme un **composant** ou un **module** particulier. Ils permettent de s'assurer que les **briques élémentaires** (ex : règles métiers) sont **correctement construites.**

Il est facile d'imaginer ce que donnerait le test et la maintenance d'un code informatique où les traitements et les règles de gestions seraient **emmêlés** : il serait très difficile d'y voir clair.

Pour cette raison, le principal prérequis aux tests unitaires est la **modularité** du code informatique. Cette modularité apporte de la robustesse à l'application développée, car il est alors possible d'agir de manière très précise sur une fraction du code source. Les tests unitaires sont encapsulés et n'utilisent pas de ressources externes. Pour toutes ces raisons, les tests unitaires sont donc généralement **simples à écrire.**

Ils doivent être exécutés après chaque changement du code source et doivent donner un retour quasi immédiat. Le but est de valider que rien n'est cassé après une nouvelle compilation. L'échec d'un test unitaire signifie la révision du code source de l'application avant d'aller plus loin.

2. Tests d'intégration

Les composants ou les modules indépendants du produit sont assemblés et sont testés **ensemble**. Ces tests permettent de rechercher des anomalies qui ne pourraient **pas** être trouvées avec des tests unitaires. Il est, à ce stade, nécessaire de travailler avec un système de **versions** afin d'organiser, de documenter et de communiquer à propos des changements et des correctifs apportés au fur et à mesure au sein du code du produit.

Les tests d'intégration ne doivent être démarrés qu'à partir du moment où les tests unitaires ont été exécutés avec succès. Poursuivez les tests d'intégration jusqu'à ce que quelque chose échoue : déterminez alors ce qui doit être ajouté ou modifié, puis réitérez le processus jusqu'à ce que tous les tests passent.

Les tests d'intégration font appel à des composants ou à des infrastructures telles que le réseau, une base de données, un annuaire d'entreprise, un système de fichiers, etc. Ils sont donc plus **complexes** à écrire. Il est important de mettre en place des logs « *les plus parlants possibles* » afin de pouvoir analyser l'origine des anomalies qui pourraient survenir, car celles-ci sont beaucoup moins évidentes à trouver que celles trouvées lors des tests unitaires.

Exemple

Quand un log vous dit « *Error 1416 : constraint not valid* », il est peu probable que vous sachiez tout de suite déterminer d'où vient le problème. Il est donc judicieux de rendre les logs plus parlants aussi tôt que possible.

Quand un test d'intégration échoue, il n'est normalement pas nécessaire d'examiner la logique « *métier* ». L'échec d'un test d'intégration signifie en effet une erreur au sein des différents environnements connectés les uns avec les autres. Plus les tests unitaires sont efficaces, plus les tests d'intégration peuvent être espacés dans le temps.

3. Tests de recette fonctionnelle

Bien que la MOE réalise évidemment des tests fonctionnels avant de livrer une nouvelle version logicielle à la MOA, la recette fonctionnelle est surtout le dernier contrôle qui permet aux utilisateurs finaux de vérifier la qualité de l'application livrée. Il est assez rare qu'une application soit mise en production après une seule recette fonctionnelle, car des anomalies sont trouvées (le contraire serait suspect). Plusieurs campagnes de tests sont donc généralement nécessaires afin de valider un passage en production.

Tous droits réservés à STUDI - Reproduction interdite

Les livrables attendus sont des bilans de tests et des **PV de recette.**

La recette fonctionnelle est exécutée par un panel d'utilisateurs métier, idéalement acquis au projet et donc moteurs. Ils suivent des scénarios de test et vérifient directement ou indirectement (via le cahier de recette) que les spécifications du produit se retrouvent bien concrétisées dans le produit testé.

Pour être pertinents et efficaces, les scénarios doivent être écrits par le client du projet numérique ou à défaut, en collaboration avec celui-ci, dès la validation des spécifications du produit. Ce ne sont pas les développeurs qui doivent écrire ces tests, car ils ne possèdent pas forcément l'expertise métier requise.

Un PV de recette doit être formalisé par le client afin de tracer le travail accompli et les résultats obtenus. Ce PV de recette peut également être une obligation contractuelle.

a. Quelques définitions

Définition **Recette Usine**

La recette usine est réalisée chez le fournisseur avant livraison à la MOA. Elle comprend les tests unitaires, les tests de validation et les tests d'intégration. Le but pour la MOE est de vérifier que son produit est de qualité, et que le client peut en faire l'installation sur ses plateformes en vue de recettes fonctionnelles (VA et VSR).

La livraison (et donc la fin de la recette usine) donne généralement lieu à un PV signé par la MOE et la MOA.

Les tests de validation sont propres à la MOE, ils permettent de vérifier que les exigences du client sont respectées, telles qu'elles ont été décrites dans les spécifications du produit.

Définition **Vérification d'Aptitude (VA)**

La vérification d'aptitude est réalisée par la MOA du projet numérique au sein d'un environnement de tests. Elle consiste à vérifier que le produit livré est **techniquement et fonctionnellement conforme.** Un PV doit être émis par la MOA à destination de la MOE, afin de prononcer l'issue de la VA (livraison acceptée ou refusée), c'est-à-dire d'acter sa **conformité**. Les délais pour prononcer la VA sont (ou doivent être) contractuels.

La mise en production progressive ou totale du produit est la suite logique d'une **VA positive.**

Définition **Vérification de Service Régulier (VSR)**

La vérification de service régulier est réalisée par la MOA du projet numérique **après la VA,** cette fois en environnement de production. La **VSR** permet donc de vérifier le comportement du produit dans son environnement réel avec les utilisateurs finaux. C'est le moment où des derniers correctifs peuvent être réclamés (sauf évidemment si le produit est couvert par un contrat de tierce maintenance applicative) mais c'est aussi le dernier moment ou des pénalités pourraient être réclamées en cas de dysfonctionnements importants et non résolus dans les temps.

Un PV doit être émis par la MOA à destination de la MOE afin de prononcer la fin de la VSR, dont le délai doit être contractuel.

Définition **MOA**

La maîtrise d'ouvrage, ou *maître d'ouvrage* est généralement abrégée **MOA**. C'est la personne **pour qui** est réalisé le projet. C'est la MOA qui est **porteuse** d'un besoin. Elle définit l'objectif du projet, son calendrier et le budget à lui consacrer. Le résultat attendu de ce projet est **l'ouvrage.**

Tous droits réservés à STUDI - Reproduction interdite

Définition **MOE**

Le maîtrise d'œuvre, ou *maître d'œuvre* est généralement abrégée **MOE**. C'est la personne ou le groupe qui assure la production d'un projet dans le respect des délais, des coûts et de la qualité attendue par le porteur du besoin. On y trouve généralement la direction technique de l'entreprise, un chef de projet, un lead développeur, ou encore un développeur directement dans le cas de projets plus petits.

Exercice : Quiz

Question 1

Quel type de test permet de vérifier le bon fonctionnement d'un composant du produit ?

- ○ Les tests unitaires
- ○ Les tests d'intégration
- ○ Les tests fonctionnels
- ○ Les tests de charge
- ○ Les tests de capacité

Question 2

Les différents modules du produit viennent tout juste d'être assemblés et peuvent être testés ensemble. Quel test allez-vous effectuer en premier ?

- ☐ Des tests unitaires
- ☐ Des tests d'intégration
- ☐ Des tests fonctionnels
- ☐ Des tests de charge
- ☐ Des tests de capacité
- ☐ Des tests de sécurité

Question 3

Les utilisateurs finaux participent aux tests. Lesquels ?

- ○ Les tests unitaires
- ○ Les tests d'intégration
- ○ Les tests fonctionnels
- ○ Les tests de montée en charge

Question 4

À partir de quel moment faut-il commencer les tests d'intégration ?

- ○ Lorsque les tests unitaires sont passés avec succès
- ○ Quand la recette fonctionnelle a été validée
- ○ Dès que les testeurs métier sont prêts

Question 5

Tous droits réservés à STUDI - Reproduction interdite

À quoi fait appel un test d'intégration ? (plusieurs réponses possibles)

- ☐ Au réseau informatique de l'organisation
- ☐ À une ou plusieurs bases de données
- ☐ À un système de fichiers
- ☐ À la logique métier développée dans le produit

Question 6

Que contient la recette usine réalisée chez un fournisseur ? (plusieurs réponses possibles)

- ☐ Les tests unitaires
- ☐ Les tests de validation
- ☐ Les tests d'intégration
- ☐ Les tests fonctionnels

Question 7

Lorsqu'un PV de VA positive est rédigé :

- ○ On installe la livraison sur l'environnement de recette ;
- ○ On installe la livraison sur l'environnement de production ;
- ○ On commence la recette fonctionnelle.

II. Tester en prenant en compte le cycle de développement

Un logiciel répond à un besoin. Il ne naît pas spontanément, sans raison. Il est généralement fabriqué par assemblage de composants existants ou développés spécifiquement, puis évolue en fonction des améliorations souhaitées ou nécessaires. Plus une erreur est détectée tôt et plus il est simple de la corriger. Il faut donc s'efforcer de tester le plus tôt possible, en s'adaptant au modèle de développement retenu pour le projet. Les modèles les plus répandus sont le cycle en V et les méthodes Agiles.

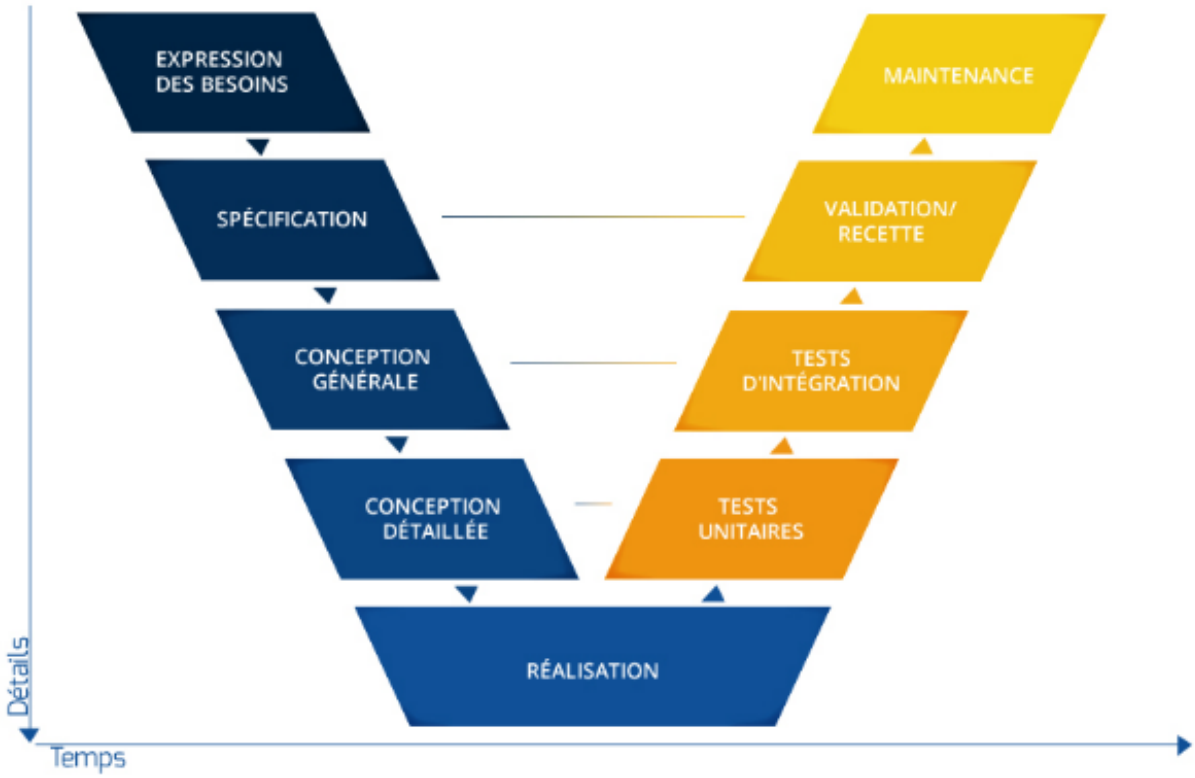
A. Cycle en V

Le cycle en V est composé d'une phase descendante (**la conception**), d'une pointe (**la réalisation**) et d'une phase montante (**la validation**). Des informations sont échangées entre les phases montantes et descendantes. À chaque étape de conception correspond une phase de test ou de validation.

Le cycle en V découpe le projet en 9 phases :

1. Expression des besoins,
2. Spécification,
3. Conception générale,
4. Conception détaillée,
5. Réalisation,
6. Tests unitaires,
7. Tests d'intégration,
8. Tests de validation / recette fonctionnelle,
9. Maintenance.

Tous droits réservés à STUDI - Reproduction interdite



Avantages

La structure en V permet d'espérer que le livrable final soit totalement conforme puisque les étapes de test sont aussi nombreuses que les étapes de réflexion. Comme il est facile de prévoir les tests à réaliser au moment où l'on conçoit une fonctionnalité ou une interface, le travail peut s'enchaîner de façon naturelle. L'avantage principal de cette méthode est qu'elle présente des gains d'échelle (planning et / ou charge) en traitant d'un bloc, l'ensemble du périmètre du projet.

Inconvénients

Il faut apporter un soin particulier à **équilibrer** le V afin que l'accent ne soit pas plus mis d'un côté que de l'autre (spécifications ou tests). Le cycle en V n'est pas adapté pour le changement. En effet, il ne prévoit pas de nouvelles décisions lors d'un cycle et les changements doivent se faire entre deux cycles en V. La communication est peu facilitée du fait du cloisonnement des rôles de chaque acteur, car ils ont chacun leur tour un rôle à jouer, mais ils ne le jouent jamais tous ensemble et communiquent principalement au travers de documents. Pour passer à l'étape suivante, il est important de terminer la précédente. Cela peut donc entraîner les mêmes dérives de planning et de coûts que dans le modèle en Cascade. La durée de ce type de projet est potentiellement longue et le résultat final peine souvent à s'adapter aux évolutions du besoin. Si la documentation liée à la conception du produit est imparfaite, le produit fini ne répondra pas entièrement aux besoins du client. La méthode ne prévoit pas d'opportunité de vérification intermédiaire.

B. Méthodes Agiles

Les méthodes de développement Agiles visent à réduire le cycle de vie des logiciels en accélérant leurs développements. Le principe général des méthodes Agiles est de développer une version minimale d'un produit (Produit Minimum Viable ou MVP), puis d'intégrer les fonctionnalités par un processus itératif, basé sur une écoute du client et des tests permanents.

L'origine de ces méthodes modernes vient du fait que les technologies évoluent très rapidement et que les clients sont souvent dans l'incapacité de définir leurs besoins de manière précise et exhaustive dès le début du projet. Les méthodes Agiles placent le client au cœur du projet et il obtient, en contrepartie d'une forte implication, une rapide mise en production d'une partie de son logiciel.

Tous droits réservés à STUDI - Reproduction interdite

Les **12 principes** généraux des méthodes Agiles sont :

1. Faire simple,
2. Privilégier la conversation face à face,
3. Construire les projets autour de personnes motivées,
4. Responsabiliser les équipes,
5. Assurer une coopération permanente entre le client et l'équipe projet,
6. Satisfaire le client en priorité,
7. Accueillir favorablement les demandes de changement,
8. Faire avancer le projet à un rythme soutenable et constant,
9. Livrer le plus souvent possible des versions opérationnelles de l'application,
10. Mesurer l'avancement du projet en matière de fonctionnalités,
11. Porter une attention continue à la qualité technique et à la conception,
12. Ajuster régulièrement les comportements et rituels pour être plus efficace.

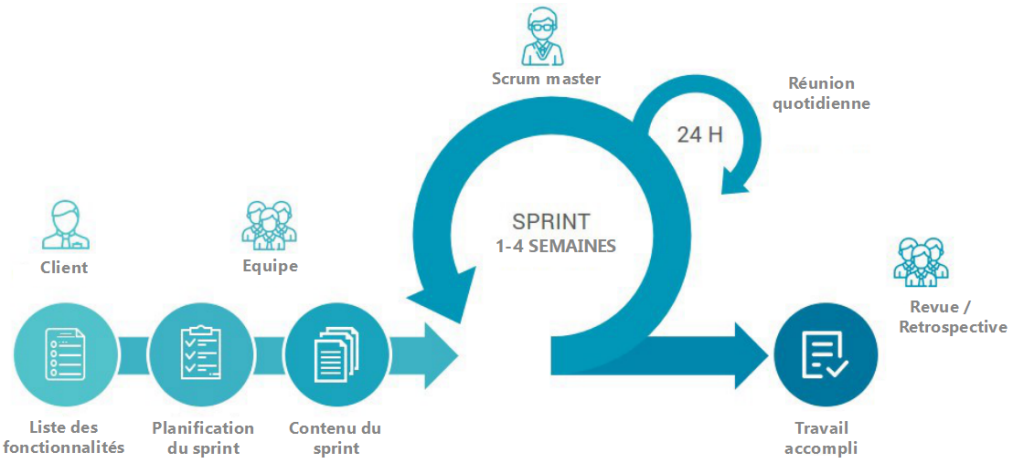
La méthodologie Agile se base sur ce principe simple : **planifier la totalité de votre projet dans les moindres détails avant de le développer est contre-productif.** L'équipe projet doit être capable de se remettre sans cesse en cause et de chercher continuellement à évoluer. Elle doit se fixer de petits objectifs, à court terme, et passer d'un objectif à l'autre jusqu'à atteindre l'objectif final.

Avantages

L'avantage majeur de l'approche Agile est sa flexibilité. Les changements et les imprévus sont pris en compte. L'équipe projet peut réagir rapidement. Une relation de confiance se tisse rapidement entre le client et l'équipe projet car la collaboration et la communication avec le client sont omniprésentes. Le client dispose d'une visibilité claire sur l'avancement du projet et peut facilement demander des ajustements. Le contrôle qualité est permanent. Le budget dépensé et le budget restants sont parfaitement identifiés étape par étape. Il est possible d'arrêter le projet si les fonds sont insuffisants : l'équipe projet aura produit le maximum de valeur avec le temps qui lui a été accordé.

Inconvénients

Les méthodes Agiles laissent peu de place à la documentation car le dialogue est privilégié. Cela peut poser des difficultés en cas de changement d'équipe projet. Le client doit être disponible et s'intéresser à son projet afin de s'assurer qu'il répondra parfaitement à ses besoins. Tous les clients n'ont pas le temps ni l'envie de s'impliquer pleinement dans la réalisation d'un projet. La méthode Agile n'est pas adaptée aux entreprises ayant une structure hiérarchique forte, à cause de son fonctionnement collaboratif. La méthode Agile permet un bon contrôle des coûts, mais elle rend très difficile la vision d'un budget pour la totalité du projet. La flexibilité a un coût que le client doit être prêt à payer.



Tous droits réservés à STUDI - Reproduction interdite

C. Préparation des tests lors des phases d'expression de besoin

Il est important de **préparer** la mise en œuvre des tests dès la phase d'expression de besoin de votre client. Cette démarche fait partie intégrante de l'assurance qualité et apporte de la confiance vis-à-vis de la recette à venir. Une recette efficace est toujours une **recette préparée** avec soin.

Il est fondamental de se poser les questions suivantes dès la phase d'expression et d'analyse du besoin :

- Va-t-on faire le bon logiciel ?
- Le logiciel sera-il bien fait ?
- Pourra t-on le tester et corriger les défauts ?

Conseil

Des besoins mal exprimés, mal compris, incomplets, ambigus ou contradictoires sont générateurs de perturbations et de problèmes en tout genre lors du développement d'un logiciel. Faites-en sorte que le besoin soit clairement et entièrement exprimé, mais qu'il soit également parfaitement compris. Réalisez des synthèses, des comptes-rendus, des maquettes. Posez des questions. Demandez systématiquement des validations à votre client et prenez garde aux non-dits. Il est important d'écrire et de partager sur ce que votre logiciel fera, mais aussi sur ce qu'il ne fera pas.

D. Préparation des tests lors de l'écriture des spécifications fonctionnelles

La phase de spécifications fonctionnelles permet de consolider le travail réalisé pendant l'expression des besoins et son analyse. Il s'agit de décrire de façon exhaustive **l'ensemble des fonctionnalités** du futur logiciel, mais aussi ses visuels et l'expérience utilisateur qu'il offrira. Décrivez précisément comment vous testerez l'ensemble des fonctionnalités attendues (séquence, résultats), mais également les éléments visuels.

Conseil

Prenez soin d'éliminer ou de préciser toutes les exigences fonctionnelles qui vous paraissent non testables. S'il n'est pas possible de tester convenablement une fonctionnalité et d'identifier clairement le résultat attendu, c'est probablement qu'elle n'a pas été suffisamment bien spécifiée ; et vous ne devriez pas la conserver en l'état.

E. Préparation des tests lors de la conception

La phase de conception est le moment où l'on passe d'un besoin métier à un système informatique. C'est le moment où l'on considère les éléments techniques nécessaires au développement et au fonctionnement du logiciel.

Conseil

Il est primordial de définir une architecture que vous maîtriserez, et qui est testable. Une architecture testable permet de minimiser le volume de tests nécessaires à la validation du logiciel dans son contexte d'exploitation réel.

C'est lors de la phase de conception que vous préparerez les **tests d'intégration.** Ces tests vous permettront d'observer les échanges au niveau des différentes interfaces (internes / externes) de votre logiciel. Par exemple, de nombreux outils sont interfacés avec un serveur de messagerie ; ce qui permet aux utilisateurs de recevoir des e-mails. Si votre logiciel est interfacé avec un serveur de messagerie, vos tests devront le prendre en compte.

Tous droits réservés à STUDI - Reproduction interdite

Conseil

Les cinq principes suivants peuvent vous aider à préparer les tests de conception :

1. Appliquez le principe de **cohésion** et de **faible couplage**. Cela signifie que les entités d'un même module devraient être fortement liées, et que les modules devraient être le plus faiblement dépendants les uns des autres.
2. Découpez les traitements les plus longs en étapes afin que des résultats intermédiaires puissent être produits et observables dans les logs.
3. Prévoyez un plan d'action pour le traitement des erreurs (Qui fait quoi ? À quel moment ?) avec une journalisation des événements. Cela vous permettra d'obtenir de précieux tableaux de bord.
4. Définissez précisément les interfaces entre vos différents modules. Cartographiez votre application. Vous pouvez pour cela vous aider d'un urbaniste et d'un intégrateur si vous en disposez.
5. Définissez un ordre d'intégration et planifiez les tests en prenant en compte les ressources humaines (besoin de compétences particulières à un moment très précis) et matérielles.

Exercice : Quiz

Question 1

À partir de quelle étape l'écriture des cas de tests est-elle possible ?

- O L'écriture des spécifications
- O La conception du logiciel
- O L'expression des besoins
- O L'intégration des composants

Question 2

Dans un cycle en V, les tests unitaires permettent de valider la bonne prise en compte de l'expression de besoin utilisateur.

- O Vrai
- O Faux

Question 3

Les méthodes Agiles permettent de libérer très tôt le client, car l'équipe de développement maîtrise parfaitement le périmètre du logiciel à développer.

- O Vrai
- O Faux

Question 4

On trouve plus de documentation dans un projet basé sur un cycle en V que dans un projet basé sur les méthodes Agiles.

- O Vrai
- O Faux

Question 5

Le cycle en V est une méthode obsolète, maintenant remplacée par les méthodes Agiles telles que Srum.

Tous droits réservés à STUDI - Reproduction interdite

- Vrai
- Faux

III. Focus sur les tests d'intégration

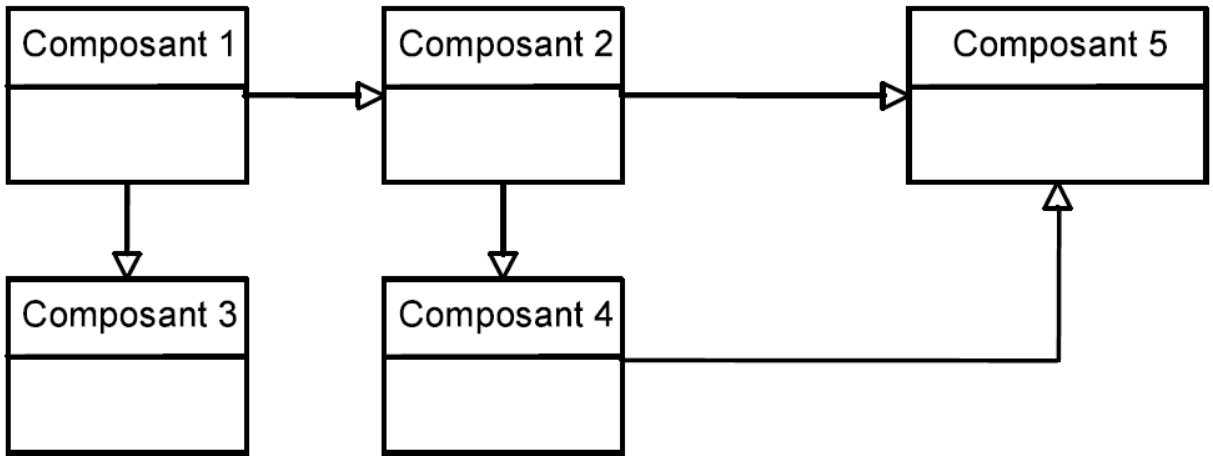
La première difficulté de l'intégration est qu'elle est totalement tributaire de la qualité des activités et des processus en amont. Selon la complexité du projet, l'intégration peut consommer de 20 % à 35 % de l'effort total de développement.

Les tests d'intégration se concentrent sur les interfaces entre les composants, les interactions entre les différentes parties du système ou les interfaces entre les systèmes.

Conseil

On ne teste pas un résultat d'ensemble lors des tests d'intégration, mais on se concentre uniquement sur les échanges au niveau des interfaces. Il est donc essentiel que vous maîtrisiez parfaitement l'architecture de votre application afin de pouvoir tester les intégrations.

Un logiciel est généralement constitué de multiples composants, et les interactions entre eux peuvent être complexes. C'est pour cette raison qu'il faut choisir l'ordre dans lequel on réalise les tests d'intégration. Dans l'exemple ci-dessous, le composant 5 utilise des données qui viennent du composant 2, qui lui-même utilise des données venant du composant 1. Il est donc judicieux de tester l'interface entre les composants 1 et 2 avant de tester l'interface entre les composants 2 et 5.



A. Stratégies du test d'intégration

Il sera plus ou moins facile de tester les interactions entre vos composants, selon la manière dont ils sont intégrés dans votre logiciel. On distingue généralement trois stratégies pour le test d'intégration :

- La stratégie du Big Bang,
- La stratégie par lots fonctionnels,
- La stratégie incrémentale.

Tous droits réservés à STUDI - Reproduction interdite

B. Stratégie du Big Bang

La stratégie du Big Bang consiste à intégrer tous les composants disponibles en même temps. Le fait que tout soit ouvert rend plus difficile la localisation d'une erreur, et pourrait même parfois vous induire en erreur quant aux pistes à explorer. Cette méthode est toutefois simple et peut s'appliquer sur de petites applications.

C. Stratégie par lots fonctionnels

La stratégie par lots fonctionnels consiste, au lieu d'ouvrir toutes les interfaces en même temps, à tester les composants selon une logique métier. Cette stratégie permet donc de tester l'interaction de composants prévus pour travailler ensemble, dans un même contexte.

On pourrait parler « *d'intégration Big Bang raisonnée* ».

D. Stratégie incrémentale

La stratégie incrémentale est la stratégie d'intégration la plus construite. Elle se base sur les dépendances entre les composants pour déterminer l'ordre dans lequel devrait être faite l'intégration.

On distingue les composants de bas niveau (sur lesquels les autres composants s'appuient) et les composants de haut niveau (ils utilisent les composants de plus bas niveau pour réaliser leurs missions et implémentent la logique de l'application).

Exemple

Dans l'exemple précédent, le composant 1 est le composant de plus bas niveau (aucun composant ne l'utilise), le composant 5 est le composant de plus haut niveau (il nécessite trois composants pour fonctionner correctement).

La première possibilité dans une intégration incrémentale est de commencer par les composants de bas niveau, puis d'intégrer au fur et à mesure les composants de plus haut niveau. Il est, de cette manière, relativement simple de trouver et de localiser les erreurs d'interaction. On ne s'appuie par conséquent que sur des composants testés au niveau unitaire et au niveau intégration, car ils sont déjà intégrés. En revanche, les tests réalisés sont très spécifiques et ne pourront plus être utilisés ultérieurement. De plus, les parties les mieux testées seront les composants de bas niveau, qui ne sont pas forcément les plus complexes.

La seconde possibilité est d'adopter une stratégie d'intégration incrémentale descendante : on intègre en premier les composants de haut niveau. L'avantage de cette stratégie est de rendre les défauts structurels très visibles, car l'architecture générale du logiciel est testée très tôt. De plus, les jeux de tests pourront resservir ultérieurement. En revanche, cette stratégie nécessite la réalisation de bouchons qui stimulent le comportement des composants absents. Certains composants de bas niveau peuvent être complexes à simuler, et la conception des bouchons peut être génératrice d'anomalies.

Exercice : Quiz

Question 1

L'intégration peut représenter plus d'un tiers de l'effort total du développement d'un logiciel.

- ○ Vrai
- ○ Faux

Question 2

Quel terme ne désigne PAS une stratégie de tests d'intégration ?

Tous droits réservés à STUDI - Reproduction interdite

- Méthode incrémentale ascendante
- Méthode incrémentale descendante
- Méthode de la régression minimale
- Méthode du big bang

Question 3

Il est nécessaire de connaître parfaitement l'architecture d'un logiciel pour pouvoir dérouler les tests fonctionnels.

- Vrai
- Faux

Question 4

La stratégie du Big Bang permet de fortement solliciter les logiciels complexes afin de détecter un maximum d'erreur.

- Vrai
- Faux

Question 5

Qu'intègre-t-on en premier dans le cadre d'une stratégie d'intégration incrémentale descendante ?

- Les composants de bas niveau
- Les composants de haut niveau

IV. Focus sur la recette fonctionnelle

Les tests de recette fonctionnelle doivent être effectués dans un environnement au plus proche possible de l'environnement cible, de façon à minimiser les erreurs liées à un environnement différent.

Les documents en entrée de la recette fonctionnelle comprennent :

- Les spécifications fonctionnelles, qui décrivent ce que l'on attend du logiciel,
- Le plan de test, qui décrit ce qu'il faut tester et comment,
- Le manuel utilisateur, qui décrit comment utiliser le logiciel.

Les tests fonctionnels visent à s'assurer que le logiciel effectue ce pourquoi il est fait.

A. Spécifications fonctionnelles

Expliquez ce que doit faire le logiciel et découpez les besoins en fonctionnalités. Vous pouvez pour cela partir de fonctionnalités principales (ou majeures) et les décliner en sous-fonctions, sans oublier de mentionner les éventuelles contraintes qui pourraient s'appliquer.

Exemple

Par exemple :

Fonctionnalité principale : traiter des dossiers de candidature.

Sous-fonction 1 : créer des dossiers *manuellement* dans l'application.

Sous-fonction 2 : importer *automatiquement* des dossiers à partir de documents Excel.

Tous droits réservés à STUDI - Reproduction interdite

Vous pouvez préciser pour chaque fonction et sous-fonction :

- L'objectif de la fonction
- La description de cette fonction
- Les contraintes à respecter
- Les règles de gestion à appliquer
- La priorité

  Et si vous en avez :

- Une maquette

Bien que ce travail puisse être long à réaliser, c'est un réel investissement susceptible de vous faire économiser énormément de temps et d'énergie sur la durée du projet.

B. Plan de test

Le plan de test doit contenir :

- Un **identifiant** qui permet d'identifier ce document sans ambiguïté, ainsi qu'un numéro de révision que l'on incrémente au fil de ses mises à jour,
- Les **références** aux documents mentionnés (normes qualité, méthodologies en vigueur, documents de spécifications, etc.),
- Une **introduction** décrivant l'objet du plan de tests,
- **L'environnement** nécessaire pour mener à bien ce plan de tests et sa planification,
- La **formation** nécessaire des utilisateurs à l'exécution de ce plan de tests,
- La répartition des **rôles** et des **responsabilités** liés à l'exécution du plan de tests,
- Les **fonctionnalités** devant être **testées** en tant qu'utilisateur,
- Les **documents produits** par le plan de test (cas de tests, jeux de valeurs, etc.),
- Les valeurs de **sortie**.

N'oubliez pas que le test d'un logiciel ne peut pas être exhaustif et que les ressources ne sont pas infinies. Vous serez donc amené à faire des choix : choix des tests, choix des environnements, choix de séquencement, choix de planification, etc., mais aussi le choix des critères qui actent la fin des tests ou leur mise en pause.

C. Manuel utilisateur

Une documentation efficace aide les utilisateurs à comprendre votre logiciel, et c'est un élément essentiel de la conduite du changement. Si complet puisse-t-il être, votre logiciel restera inutile si les utilisateurs le boudent !

Voici un guide qui vous permettra de concevoir une documentation efficace :

Comprenez la raison d'être de votre document.

Bien que la principale raison de réaliser de la documentation informatique soit d'aider les utilisateurs à comprendre comment utiliser un logiciel, il existe plusieurs autres raisons toutes aussi importantes. Une documentation solide permet de faciliter la commercialisation d'un logiciel, sa maintenance, son exploitation, renforcer l'image de marque de l'entreprise, réduire les coûts d'assistance aux utilisateurs.

Comprenez votre public cible.

Dans de nombreux cas, les utilisateurs ont peu ou pas de connaissances informatiques, au-delà des tâches qu'ils accomplissent avec leurs logiciels habituels. Cependant, il y a plusieurs approches pour apporter des solutions à leurs besoins avec votre documentation. Prenez en compte les intitulés de poste des utilisateurs potentiels du logiciel. Un utilisateur administrateur système est éventuellement un expert pouvant travailler avec plusieurs applications différentes. Mais un commis à la saisie de données, peut n'être seulement compétent qu'avec le logiciel

Tous droits réservés à STUDI - Reproduction interdite

qu'il ou elle emploie pour l'entrée de données. Envisagez de discuter avec votre public cible pour mieux cerner leur environnement, leur métier, leurs connaissances, leurs besoins, leurs inquiétudes. Ces réponses seront précieuses pour la rédaction de votre guide utilisateur.

Faites-vous une idée de la forme que la documentation doit prendre.

La documentation du logiciel pour les utilisateurs peut être publiée sous plusieurs formes différentes ou n'être éditée que sous une seule, comme des manuels imprimés, des documents informatiques, une aide en ligne, une chatbox. Chaque format est créé pour montrer à l'utilisateur comment utiliser les différentes fonctionnalités du logiciel. Cela peut prendre la forme d'un tutoriel ou d'un guide pas à pas. Certains fichiers d'aide et aides en ligne peuvent avoir des vidéos de démonstration, du texte et des graphiques.

Faites en sorte que ce soit agréable !

Nous avons tous été un jour à la place d'un utilisateur à la recherche d'une aide sur un logiciel. Qu'avez-vous pensé de cette expérience ? Que feriez-vous afin de rendre votre propre documentation agréable ?

Les options sont nombreuses, faites appel à votre bon sens et surtout, n'hésitez pas à tester votre documentation auprès d'un panel d'utilisateurs cibles !

Exercice : Quiz

Question 1

Afin de gagner en fiabilité, le plan de tests doit être modifié le moins souvent possible.

- O Vrai
- O Faux

Question 2

L'environnement de l'utilisateur doit être décrit dans un plan de tests fonctionnel.

- O Vrai
- O Faux

Question 3

Le manuel utilisateur est livré au moment de la mise en production du logiciel.

- O Vrai
- O Faux

Question 4

Une documentation logicielle peut être testée.

- O Vrai
- O Faux

Question 5

Un bon plan de tests est un plan exhaustif.

- O Vrai
- O Faux

Tous droits réservés à STUDI - Reproduction interdite

VII. Essentiel

Nous avons vu ensemble qu'il n'est pas possible de tout tester. Les ressources humaines et matérielles sont limitées. Par contre, il est important de planifier une stratégie de tests cohérente le plus tôt possible dans la vie de votre projet. Qu'allez-vous tester ? À quel moment ? Par qui ? Comment prendrez-vous en compte les retours ? Quels traitements allez-vous appliquer ?

Le contenu de ce cours a pour objectif de vous introduire à l'activité de testing et vous fournir des conseils pour mieux réussir ! N'oubliez pas qu'il est judicieux d'écrire les tests dès lors que vous recueillez et analysez les besoins de votre client.

VIII. Auto-évaluation

A. Exercice :

Vous venez de sortir une nouvelle application web embarquant de nombreuses fonctionnalités dédiées à la sécurité. Vous souhaitez tester les mécanismes d'inscription à votre application, de login / logout et de double authentification. La double authentification est demandée à chaque fois que l'utilisateur se connecte via un nouvel appareil, et dans tous les cas, 72 H après sa dernière connexion s'il ne s'est pas connecté entre temps. Les comptes sont automatiquement bloqués pendant 48 H dès qu'un mot de passe erroné a été saisi 3 fois en moins d'une heure.

Question 1

Comment testez-vous la fonctionnalité d'inscription à votre application ?

Question 2

Comment testez-vous la fonctionnalité de double authentification ?

Question 3

Comment testez-vous la fonctionnalité de login ?

B. Test

Vous venez de sortir une nouvelle application web embarquant de nombreuses fonctionnalités dédiées à la sécurité. Vous souhaitez tester les mécanismes d'inscription à votre application, de login / logout et de double authentification. La double authentification est demandée à chaque fois que l'utilisateur se connecte via un nouvel appareil, et dans tous les cas, 72 H après sa dernière connexion s'il ne s'est pas connecté entre temps. Les comptes sont automatiquement bloqués pendant 48 H dès qu'un mot de passe erroné a été saisi 3 fois en moins d'une heure.

Exercice : Quiz

Question 1

Laquelle de ces vérifications est réalisée en première ?

- ○ La VA
- ○ La VSR

Question 2

Qui écrit le cahier de recette fonctionnelle ?

- ○ Le client
- ○ Le développeur
- ○ Le testeur

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Vous souhaitez développer un nouveau site internet et vous confiez ce projet à un prestataire externe. Qui êtes-vous ?

- La MOE
- La MOA

Question 4

La stratégie de tests Big Bang permet d'exécuter très rapidement l'ensemble des tests unitaires.

- Vrai
- Faux

Question 5

On veut tester différents composants d'un logiciel. On décide de faire participer un utilisateur motivé. On lui demande de se connecter à l'application, d'éditer son compte, d'enregistrer des modifications, puis de tout remettre à zéro, de fermer la fenêtre et de vérifier la présence d'un e-mail dans sa boîte de réception. De quel type de stratégie de test d'intégration s'agit-il ?

- Stratégie du Big Bang
- Stratégie par lots fonctionnels
- Stratégie incrémentale
- Stratégie incrémentale descendante

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Quel type de test permet de vérifier le bon fonctionnement d'un composant du produit ?

- (●) Les tests unitaires
- ( ) Les tests d'intégration
- ( ) Les tests fonctionnels
- ( ) Les tests de charge
- ( ) Les tests de capacité

Les tests unitaires permettent de vérifier le bon fonctionnement d'une partie précise du produit, comme un composant ou un module particulier. Ils permettent aussi de s'assurer que les briques élémentaires (ex : règles métiers) sont correctement construites.

Question 2

Les différents modules du produit viennent tout juste d'être assemblés et peuvent être testés ensemble. Quel test allez-vous effectuer en premier ?

- [ ] Des tests unitaires
- [x] Des tests d'intégration
- [ ] Des tests fonctionnels
- [ ] Des tests de charge
- [ ] Des tests de capacité
- [ ] Des tests de sécurité

Les composants ou les modules indépendants du produit sont assemblés et doivent être testés ensemble grâce aux tests d'intégration.

Question 3

Les utilisateurs finaux participent aux tests. Lesquels ?

- ( ) Les tests unitaires
- ( ) Les tests d'intégration
- (●) Les tests fonctionnels
- ( ) Les tests de montée en charge

Seuls les tests fonctionnels sont réalisés par les utilisateurs finaux. Les autres tests mentionnés sont des tests techniques, généralement automatisés.

Question 4

À partir de quel moment faut-il commencer les tests d'intégration ?

- (●) Lorsque les tests unitaires sont passés avec succès
- ( ) Quand la recette fonctionnelle a été validée
- ( ) Dès que les testeurs métier sont prêts

Tous droits réservés à STUDI - Reproduction interdite

Les tests d'intégration ne doivent être démarrés qu'à partir du moment où les tests unitaires ont été exécutés avec succès.

Question 5

À quoi fait appel un test d'intégration ? (plusieurs réponses possibles)

- [x] Au réseau informatique de l'organisation
- [x] À une ou plusieurs bases de données
- [x] À un système de fichiers
- [ ] À la logique métier développée dans le produit

Les tests d'intégration font appel à des composants ou à des infrastructures tels que le réseau, une base de données, un annuaire d'entreprise, un système de fichiers, etc. L'échec d'un test d'intégration signifie en effet une erreur au sein des différents environnements connectés les uns avec les autres.

Question 6

Que contient la recette usine réalisée chez un fournisseur ? (plusieurs réponses possibles)

- [x] Les tests unitaires
- [x] Les tests de validation
- [x] Les tests d'intégration
- [ ] Les tests fonctionnels

La recette usine est réalisée chez le fournisseur avant livraison à la MOA. Elle comprend les tests unitaires, les tests de validation et les tests d'intégration. Le but pour la MOE est de vérifier que son produit est de qualité, et que le client peut en faire l'installation sur ses plateformes en vue de recettes fonctionnelles.

Question 7

Lorsqu'un PV de VA positive est rédigé :

- ○ On installe la livraison sur l'environnement de recette ;
- ◉ On installe la livraison sur l'environnement de production ;
- ○ On commence la recette fonctionnelle.

La vérification d'aptitude est réalisée au sein d'un environnement de tests. La mise en production progressive ou totale du produit est la suite logique d'une VA positive.

Exercice p. 11 Solution n°2

Question 1

À partir de quelle étape l'écriture des cas de tests est-elle possible ?

- ○ L'écriture des spécifications
- ○ La conception du logiciel
- ◉ L'expression des besoins
- ○ L'intégration des composants

La préparation des tests peut (et devrait) être faite en même temps que l'expression et l'analyse des besoins. Le plus tôt est le mieux !

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Dans un cycle en V, les tests unitaires permettent de valider la bonne prise en compte de l'expression de besoin utilisateur.

- ○ Vrai
- ◉ Faux

Les tests unitaires permettent de s'assurer de la conformité de ce qui a été développé, par rapport aux aspects de conception détaillée.

Question 3

Les méthodes Agiles permettent de libérer très tôt le client, car l'équipe de développement maîtrise parfaitement le périmètre du logiciel à développer.

- ○ Vrai
- ◉ Faux

Les méthodes Agiles placent le client au cœur du projet et il obtient, en contrepartie d'une forte implication, une rapide mise en production d'une partie de son logiciel. Le succès du projet dépend donc grandement de l'implication du client !

Question 4

On trouve plus de documentation dans un projet basé sur un cycle en V que dans un projet basé sur les méthodes Agiles.

- ◉ Vrai
- ○ Faux

Les méthodes Agiles laissent peu de place à la documentation, car le dialogue est privilégié, alors que tout doit être planifié (et écrit) à l'avance lorsque l'on travaille au sein d'un cycle en V.

Question 5

Le cycle en V est une méthode obsolète, maintenant remplacée par les méthodes Agiles telles que Srum.

- ○ Vrai
- ◉ Faux

Le cycle en V est une méthode de développement qui a fait ses preuves et qui est encore beaucoup utilisée. Il n'est pas toujours possible de travailler avec une méthode Agile, et le cycle en V se montre efficace.

Exercice p. 13 Solution n°3

Question 1

L'intégration peut représenter plus d'un tiers de l'effort total du développement d'un logiciel.

- ◉ Vrai
- ○ Faux

Selon la complexité du projet, l'intégration peut consommer de 20 % à 35 % de l'effort total de développement.

Question 2

Quel terme ne désigne PAS une stratégie de tests d'intégration ?

- ○ Méthode incrémentale ascendante
- ○ Méthode incrémentale descendante

Tous droits réservés à STUDI - Reproduction interdite

- [x] Méthode de la régression minimale
- [ ] Méthode du big bang

La méthode de la régression minimale n'est pas une stratégie de test d'intégration.

Question 3

Il est nécessaire de connaître parfaitement l'architecture d'un logiciel pour pouvoir dérouler les tests fonctionnels.

- [ ] Vrai
- [x] Faux

La connaissance de l'architecture d'un logiciel est nécessaire au bon déroulement des tests d'intégration. La recette fonctionnelle vient après l'intégration.

Question 4

La stratégie du Big Bang permet de fortement solliciter les logiciels complexes afin de détecter un maximum d'erreur.

- [ ] Vrai
- [x] Faux

La stratégie du Big Bang consiste à intégrer tous les composants disponibles en même temps. Cette méthode est simple et s'applique sur de petites applications, car il devient très difficile de trouver l'origine d'une erreur au sein d'un logiciel complexe.

Question 5

Qu'intègre-t-on en premier dans le cadre d'une stratégie d'intégration incrémentale descendante ?

- [ ] Les composants de bas niveau
- [x] Les composants de haut niveau

L'avantage d'intégrer les composants de haut niveau en premier est de rendre les défauts structurels visibles très tôt.

Exercice p. 16 Solution n°4

Question 1

Afin de gagner en fiabilité, le plan de tests doit être modifié le moins souvent possible.

- [ ] Vrai
- [x] Faux

C'est le contraire ! Il est crucial de mettre à jour le plan de tests aussi souvent que nécessaire afin qu'il soit pertinent.

Question 2

L'environnement de l'utilisateur doit être décrit dans un plan de tests fonctionnel.

- [x] Vrai
- [ ] Faux

Il est important de déterminer les conditions techniques qui permettent aux utilisateurs de démarrer leurs tests (utilisation de matériel spécifique, adresse de connexion au serveur de recette, versions logicielles à utiliser, etc.)

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Le manuel utilisateur est livré au moment de la mise en production du logiciel.

- ○ Vrai
- ◉ Faux

Le manuel utilisateur (ou sa mise à jour) devrait être livré avant le démarrage de la recette fonctionnelle. Cela permettrait aux testeurs de se repérer dans l'application.

Question 4

Une documentation logicielle peut être testée.

- ◉ Vrai
- ○ Faux

La documentation peut en effet être testée. Si vous estimez qu'un administrateur nouvellement nommé sur votre projet doit pouvoir réaliser une opération de maintenance standard en moins de 20 minutes grâce à votre documentation, il suffit de lui demander d'en réaliser une et d'observer le temps nécessaire pour conduire l'opération, en plus de s'assurer qu'il a bien réussi sa mission. Votre documentation a été testée et validée.

Question 5

Un bon plan de tests est un plan exhaustif.

- ○ Vrai
- ◉ Faux

Le test d'un logiciel ne peut pas être exhaustif et les ressources sont limitées, c'est pourquoi vous devrez toujours faire des choix. Vous ne pourrez, par exemple, jamais tester votre application mobile sur tous les modèles de smartphone existants dans le monde. Le plan de tests exhaustif n'existe pas.

Exercice p. Solution n°5

Testez l'inscription en utilisant les navigateurs web recommandés par votre organisation. Vérifiez le caractère obligatoire ou facultatif de chaque champ. Vérifiez qu'il soit impossible de valider son inscription s'il manque au moins une information obligatoire. Testez le format des différents champs proposés (par exemple, en inscrivant uniquement des chiffres dans le champ « *prénom* », en oubliant le signe « @ » dans le champ adresse e-mail, en écrivant des lettres dans le « *numéro de téléphone* ». Testez également le nombre de caractères autorisés sur chaque champ (minimum / maximum). Une fois inscrit, essayez de vous connecter en utilisant volontairement un mauvais mot de passe, puis le bon.

Exercice p. Solution n°6

Vérifiez que la double authentification vous soit demandée lors de votre toute première connexion. Utilisez un autre ordinateur le même jour et connectez-vous. Vérifiez que la double authentification soit encore demandée. Déconnectez-vous. Reconnectez-vous. La double authentification ne devrait pas être requise.

Attendez 3 jours ou demandez aux administrateurs techniques de modifier en base de données la date de votre dernière connexion. Connectez-vous à l'aide d'un équipement déjà utilisé et vérifiez que la double authentification vous soit demandée.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. Solution n°7

Essayez de vous connecter en utilisant 2 mots de passe erronés, puis en saisissant le bon. Attendez une heure (ou demandez à l'administrateur de modifier votre heure de dernière connexion), et essayez maintenant d'entrer 3 fois un mauvais mot de passe. Vérifiez que votre compte soit bien bloqué pour une durée de 48 H.

Exercice p. 17 Solution n°8

Question 1

Laquelle de ces vérifications est réalisée en première ?

- (●) La VA
- ( ) La VSR

La vérification de service régulier est réalisée par la MOA du projet numérique après la VA, et cette fois en environnement de production. La VSR permet donc de vérifier le comportement du produit dans son environnement réel avec les utilisateurs finaux.

Question 2

Qui écrit le cahier de recette fonctionnelle ?

- (●) Le client
- ( ) Le développeur
- ( ) Le testeur

Pour être pertinents et efficaces, les scénarios doivent être écrits par le client du projet numérique ou à défaut, en collaboration avec celui-ci dès la validation des spécifications du produit. Ce ne sont pas les développeurs qui doivent écrire ces tests, car ils ne possèdent pas l'expertise métier requise. Ce ne sont pas non plus les testeurs, car une prise de recul est nécessaire. Et les tests pourraient être sous-traités.

Question 3

Vous souhaitez développer un nouveau site internet et vous confiez ce projet à un prestataire externe. Qui êtes-vous ?

- ( ) La MOE
- (●) La MOA

Vous êtes le maître d'ouvrage. C'est vous qui portez le besoin et définissez les règles à suivre pour le déroulement du projet.

Question 4

La stratégie de tests Big Bang permet d'exécuter très rapidement l'ensemble des tests unitaires.

- ( ) Vrai
- (●) Faux

La stratégie du Big Bang consiste à intégrer tous les composants disponibles en même temps.

Question 5

Tous droits réservés à STUDI - Reproduction interdite

On veut tester différents composants d'un logiciel. On décide de faire participer un utilisateur motivé. On lui demande de se connecter à l'application, d'éditer son compte, d'enregistrer des modifications, puis de tout remettre à zéro, de fermer la fenêtre et de vérifier la présence d'un e-mail dans sa boîte de réception. De quel type de stratégie de test d'intégration s'agit-il ?

- ○ Stratégie du Big Bang
- ◉ Stratégie par lots fonctionnels
- ○ Stratégie incrémentale
- ○ Stratégie incrémentale descendante

La stratégie décrite ici est clairement orientée « *connexion / édition de profil* ». Il s'agit d'une stratégie par lot fonctionnel.

Tous droits réservés à STUDI - Reproduction interdite